

ЦИКЛ WHILE В PYTHON.



Немного вспомним прошлый урок



Ввод и вывод данных в Python

print() — это команда, которая даёт компьютеру инструкцию: “Выведи что-то на экран”.

```
print("Hello World")  
#Программа выведет на экран: "Hello World"
```

input() — команда, которая даёт возможность пользователю(нам) ввести данные в консоль.

```
name = input("Как тебя зовут? ")  
print("Привет, " + name + "!")
```

Переменная — это способ сохранить какую-то информацию в программе, чтобы можно было её потом использовать, изменить или показать.

```
name = "Маша"
```

name — имя переменной, «**=**» — знак присваивания, «**Маша**» — значение, которое мы сохраняем

Основные типы данных в Python

- **Строка (str)** — это текст (от слова string - строка)

Это всё, что находится внутри кавычек: " , или ' '. Даже числа в кавычках считаются строкой!

```
1 name_1="Маша"  
2 name_2='Мария'  
3 age="12"
```

- **Числа**

int (от слова integer — целое число):

```
age=12
```

float (число с запятой, но в коде — с точкой):

```
height=155.5
```

- **Логический тип (bool)** — это правда или ложь

```
is_raining = True  
is_sunny = False
```

В Python есть два логических значения: **True (правда)** и **False (ложь)**. Они пишутся с заглавной буквы!

Математические операции

- **+** — сложение
- **-** — вычитание
- ***** — умножение
- **/** — деление с остатком(результатом деления всегда будет дробное число (тип float), даже если всё делится без остатка)
например $2/4=0,5$ $2/2=1.00$
- **//** — деление без остатка(результатом деления всегда будет дробное число (тип float), даже если всё делится без остатка)
например $2/4=0$ $2/2=1$ то есть берем всё что до запятой

Преобразование типов данных

Такие команды «переводчики» есть у каждого типа данных : **int()**, **float()**, **str()** и т.д.

```
1 age = input("Сколько тебе лет?")
2 age = int(age)
3 print(age + 5)
```

input() **ВСЕГДА** принимает значение **str** строки, даже если вы ввели число

А со строками **str** **НЕЛЬЗЯ** производить **математические операции (+ - * /)**

Условные конструкции

Это способ заставить программу **делать определенные действия** если **условие выполняется (Истина)** или **не выполняется (Ложь)**

Ключевая структура: **if-elif-else**

Синтаксис конструкции if-elif-else

```
if условие1:  
    # блок кода 1 (выполняется, если условие1 – True)  
elif условие2:  
    # блок кода 2 (если условие1 – False, но условие2 – True)  
elif условие3:  
    # блок кода 3 (если первые два – False, а это – True)  
...  
else:  
    # блок кода по умолчанию (если ни одно из условий не подошло)
```

Операции сравнения:

- **>** (больше)
- **<** (меньше)
- **>=** (больше или равно)
- **<=** (меньше или равно)
- **==** (равно)
- **!=** (не равно)

Использование логических операторов

and — логическое «И»: условие истинно, если истинны оба выражения.

$$1) \quad \begin{array}{c} 1 > 0 \\ \text{И} \end{array} \text{ and } \begin{array}{c} 2 > 1 \\ \text{И} \end{array} = \text{И}$$

$$3) \quad \begin{array}{c} 1 > 2 \\ \text{Л} \end{array} \text{ and } \begin{array}{c} 1 > 0 \\ \text{И} \end{array} = \text{Л}$$

$$2) \quad \begin{array}{c} 1 > 0 \\ \text{И} \end{array} \text{ and } \begin{array}{c} 0 > 1 \\ \text{Л} \end{array} = \text{Л}$$

$$4) \quad \begin{array}{c} 1 > 2 \\ \text{Л} \end{array} \text{ and } \begin{array}{c} 0 > 1 \\ \text{Л} \end{array} = \text{Л}$$

Использование логических операторов

or — логическое «ИЛИ»: условие истинно, если истинно хотя бы одно выражение.

$$\begin{array}{ll} 1) \quad \underset{\text{И}}{1 > 0} \text{ or } \underset{\text{И}}{2 > 1} = \text{И} & 3) \quad \underset{\text{Л}}{1 > 2} \text{ or } \underset{\text{И}}{1 > 0} = \text{И} \\ 2) \quad \underset{\text{И}}{1 > 0} \text{ or } \underset{\text{Л}}{0 > 1} = \text{И} & 4) \quad \underset{\text{Л}}{1 > 2} \text{ or } \underset{\text{Л}}{0 > 1} = \text{Л} \end{array}$$

Использование логических операторов

not — логическое «НЕ»: инвертирует результат (истинное становится ложным и наоборот).

$$1) \quad \text{not } 1 > 0 = \text{False}$$

False

$$2) \quad \text{not } 0 > 1 = \text{True}$$

True

Что такое цикл?

Цикл – выполнение действия несколько раз автоматически.

for используется, когда мы знаем, **сколько раз нужно повторить действие**, или хотим пройти по набору элементов.

Простая структура цикла **for**:

```
for переменная in последовательность:  
    действия внутри цикла
```

Цикл for

range() —это функция, которая создаёт последовательность чисел.

```
for i in range(1, 6):  
    print(i)
```

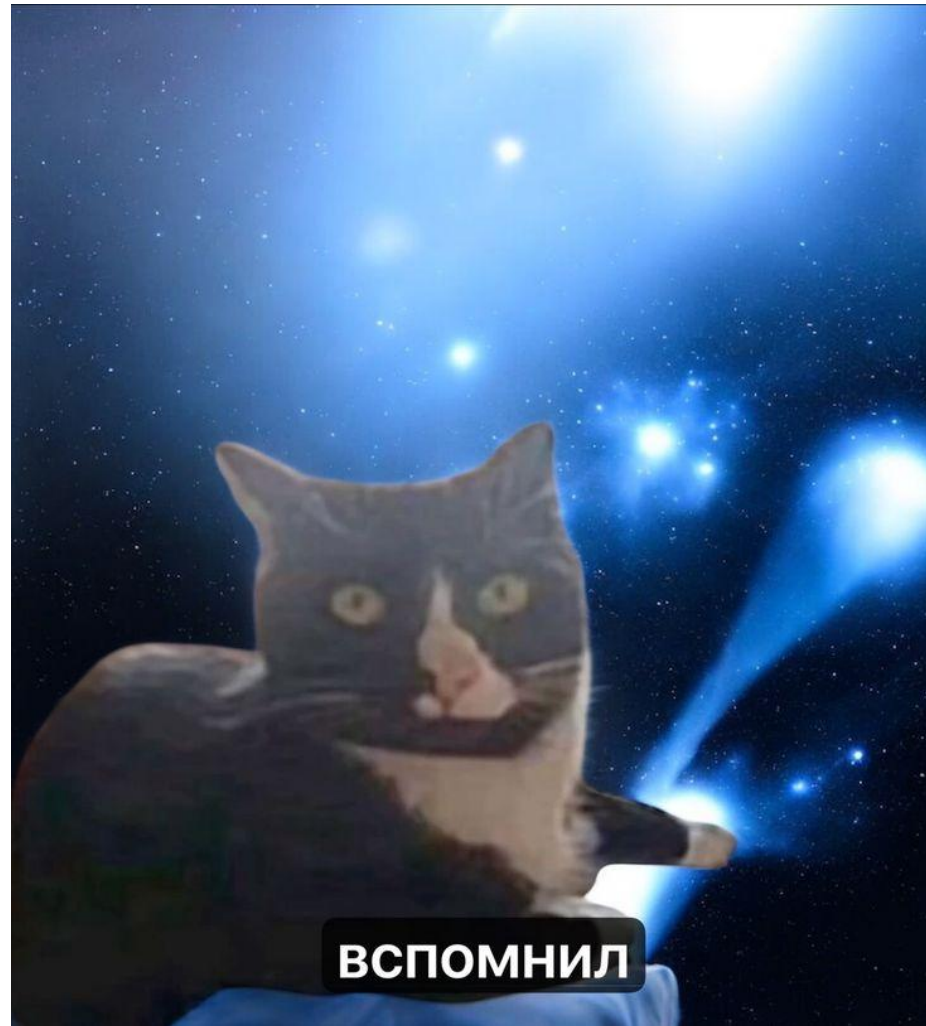
range(СТАРТ, СТОП, ШАГ)

«СТАРТ» — с какого числа начать (по умолчанию 0)

«СТОП» — до какого числа идти (не включая его!)

«ШАГ» — на сколько увеличивать (или уменьшать) каждое следующее число (по умолчанию +1)

Переходим к теме урока

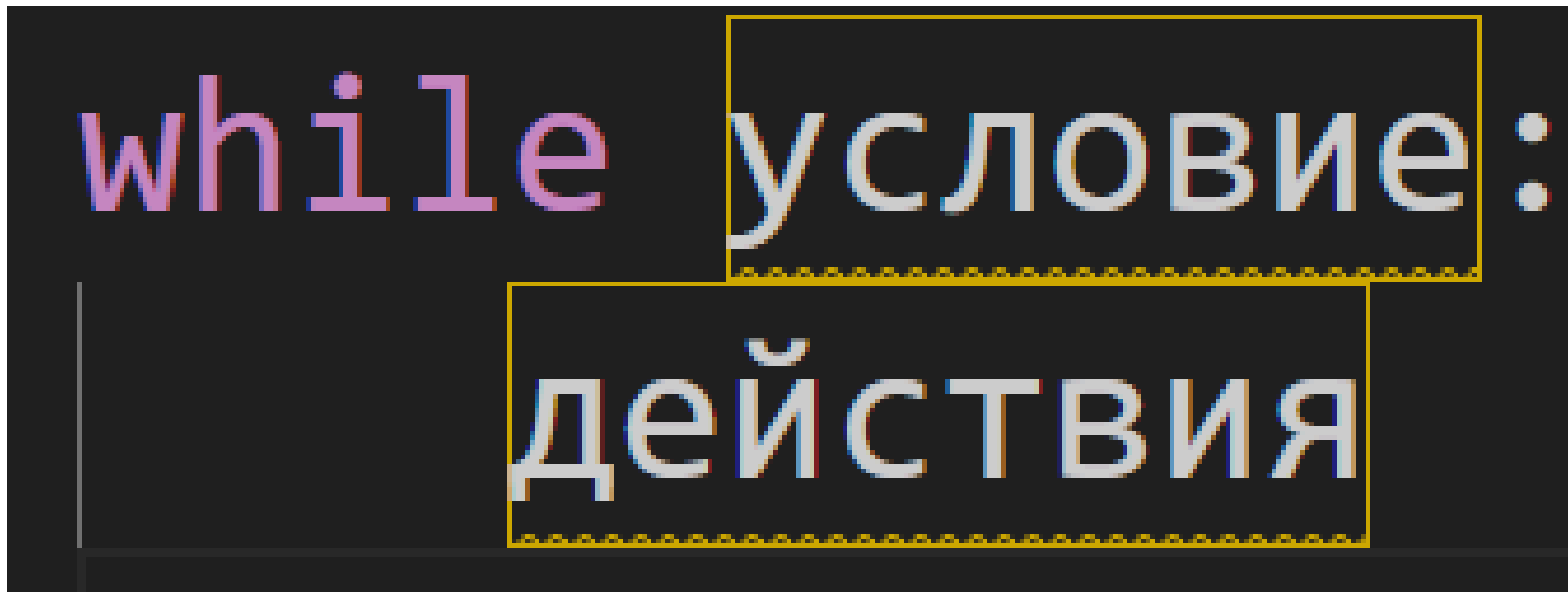


Что такое цикл while?

Цикл **while** выполняет действия пока выполняется условие.

Мы не всегда знаем заранее, сколько раз придётся повторить действия — цикл сам будет проверять условие каждый раз перед выполнением.

Ключевая структура **while**:



The diagram illustrates the structure of a while loop. It features the word "while" in pink, followed by "условие:" in white. A yellow dashed box encloses the word "условие:". Below this, the word "действия" is written in white, and another yellow dashed box encloses it. A vertical line is positioned to the left of the "действия" box, indicating the start of the loop body.

```
while условие:  
    действия
```

Немного практики

```
1 i = 1
2
3 while i <= 5:
4     print(i)
5     i += 1
```

```
PS C:\Users\Home
1
2
3
4
5
PS C:\Users\Home
```

Потом попробуйте написать **то же самое** с помощью цикла **for**

Важные правила для цикла while:

- Условие должно изменяться!

Иначе получится бесконечный цикл, который никогда не остановится.

- Контролируйте переменные внутри цикла.



Как ещё можно использовать while

Программа которая ждет ответ «да»

```
answer = ""

while answer != "да":
    answer = input("Ты выучил цикл while? (да/нет): ")

print("Отлично! Переходим к следующей теме!")
```

```
Python 3.6.1 Shell (base) C:\code\projects\py>
Ты выучил цикл while? (да/нет): нет
Ты выучил цикл while? (да/нет): нет
Ты выучил цикл while? (да/нет): нет
Ты выучил цикл while? (да/нет): да
Отлично! Переходим к следующей теме!
```

Программа «Угадай число»

```
1 secret_number = 4
2 print("Привет! Я загадал число от 1 до 10. Попробуй угадать!")
3 guess = 0
4 while guess != secret_number:
5     guess = int(input("Введи своё предположение: "))
6     if guess == secret_number:
7         print("Поздравляю! Ты угадал!")
8     else:
9         print("Неправильно. Попробуй ещё раз.")
```

```
Привет! Я загадал число от 1 до 10. Попробуй угадать!
Введи своё предположение: 8
Неправильно. Попробуй ещё раз.
Введи своё предположение: 0
Неправильно. Попробуй ещё раз.
Введи своё предположение: 1
Неправильно. Попробуй ещё раз.
Введи своё предположение: 4
Поздравляю! Ты угадал!
```

Вечный цикл и оператор break

Когда мы пишем цикл **while**, он будет выполняться пока условие истинно (пока результат проверки условия — **True**).

```
while True:  
    print("Этот цикл никогда не остановится сам!")
```

*чтобы завершить этот цикл нажмите **Ctrl C** в терминале

Вечный цикл и оператор break

Но иногда можно специально или по ошибке написать такой цикл, который никогда не остановится сам. Такой цикл называется вечным (или бесконечным).

Бесконечные циклы могут "зависнуть" и **заблокировать работу программы**. Поэтому с ними нужно быть осторожными!



Как остановить вечный цикл?

Чтобы выйти из бесконечного или любого другого цикла в нужный момент, используют специальную команду — **break**.

Команда `break` прерывает выполнение цикла и сразу выходит из него.

```
17 while True:
18     text = input("Напиши 'стоп', чтобы выйти: ")
19     if text == "стоп":
20         break
21     print("Ты написал:", text)
```

Здесь программа только проверяет, чётное ли число. Если да — выводит сообщение, если нет — программа ничего не выводит.

Очень важно

break выходит только из **одного цикла**, в котором он находится.

Если цикл вложенный (один цикл внутри другого), **break** остановит **только внутренний цикл**.

Где применяют вечные циклы и break?

Бесконечные циклы с **break** часто используют, когда не знаем заранее, сколько раз нужно повторять действия.

Например:

- Ждать правильного ответа от пользователя.
- Работать с данными, пока пользователь не скажет «хватит».
- В играх — чтобы программа работала, пока игрок не проиграет.

Что сравниваем	for	while
Количество повторений	Заранее известно	Может быть неизвестно
Условие остановки	Проход по всем элементам диапазона (<code>range()</code>)	Проверка логического условия (<code>True/False</code>)
Когда использовать	Нужно пройти все шаги, например, от 1 до 10	Нужно повторять, пока условие выполняется
Риск вечного цикла	Нет (если правильно указан <code>range</code>)	Есть, если забыть изменить условие
Где удобно использовать	Счётчики, таймеры, перебор чисел	Ожидание правильного ответа, работа с ошибками

Немного практики 1

Теперь попробуйте написать код который выводит **таблицу умножения на 5** с помощью цикла **while** или **for** на ваш выбор

```
while условие:  
    действия
```

```
for переменная in последовательность:  
    действия внутри цикла
```

Генерация случайных чисел

Библиотека **random** – вспомогательные функции, которые могут генерировать случайные значения.

Библиотеки подключаются один раз в начале кода на самой верхней строке

import random — команда, которая подключает в нашу программу модуль random.

A screenshot of a code editor with a dark background. At the top, there are three tabs: 'Settings', 'lesson5.py', and 'lesson6_1.py'. The 'lesson6_1.py' tab is active and has a white dot on its right side. Below the tabs, the text 'lesson6_1.py' is visible in the left margin. The main area of the editor shows a single line of code: '1 import random'. The number '1' is in a light gray font, 'import' is in a pinkish-purple font, and 'random' is in a green font.

Генерация случайных чисел

Функции библиотеки прописываются через точку:

библиотека.функция

`random.randint(начальное число, конечное число)`

Здесь `.randint` это функция библиотеки `random` которая выбирает случайное число в пределах `начального` и `конечного` числа включая их

```
1 import random
2
3 random_number = random.randint(1, 100)
4 print("Сегодня ты съешь:", random_number, "пельменей")
```

```
PS C:\Users\Home\VS code Projects\pr
Сегодня ты съешь: 58 пельменей
Сегодня ты съешь: 24 пельменей
```

Вы можете написать свой креативный текст и выбрать свои числа 😊

Немного практики 2

Напишем программу «Гонка черепах»

```
1  import random
2
3  turtle = 0
4  finish = 20
5
6  print("Черепашка ползёт к финишу (20 шагов)!")
7
8  while turtle < finish:
9      step = random.randint(1, 3)
10     turtle += step
11     print(f"Черепашка проползла {step} шагов. Всего: {turtle}")
12
13  print("Черепашка финишировала!")
```

Немного практики 2

```
3 turtle1 = 0
4 turtle2 = 0
5 finish = 20
6
7 print("Черепашка ползёт к финишу (20 шагов)!")
8
9 while turtle1 < finish or turtle2 < finish:
10     step1 = random.randint(1, 3)
11     turtle1 += step1
12
13     step2 = random.randint(1, 3)
14     turtle2 += step2
15
16     print(f"Черепашка 1 проползла {step1} шагов. Всего: {turtle1}")
17     print(f"Черепашка 2 проползла {step2} шагов. Всего: {turtle2}")
18
19 if turtle1 > turtle2:
20     print("Победила первая черепашка")
21 elif turtle2 > turtle1:
22     print("Победила вторая черепашка")
23 else:
24     print("Победила дружба!")
```

Немного практики 3

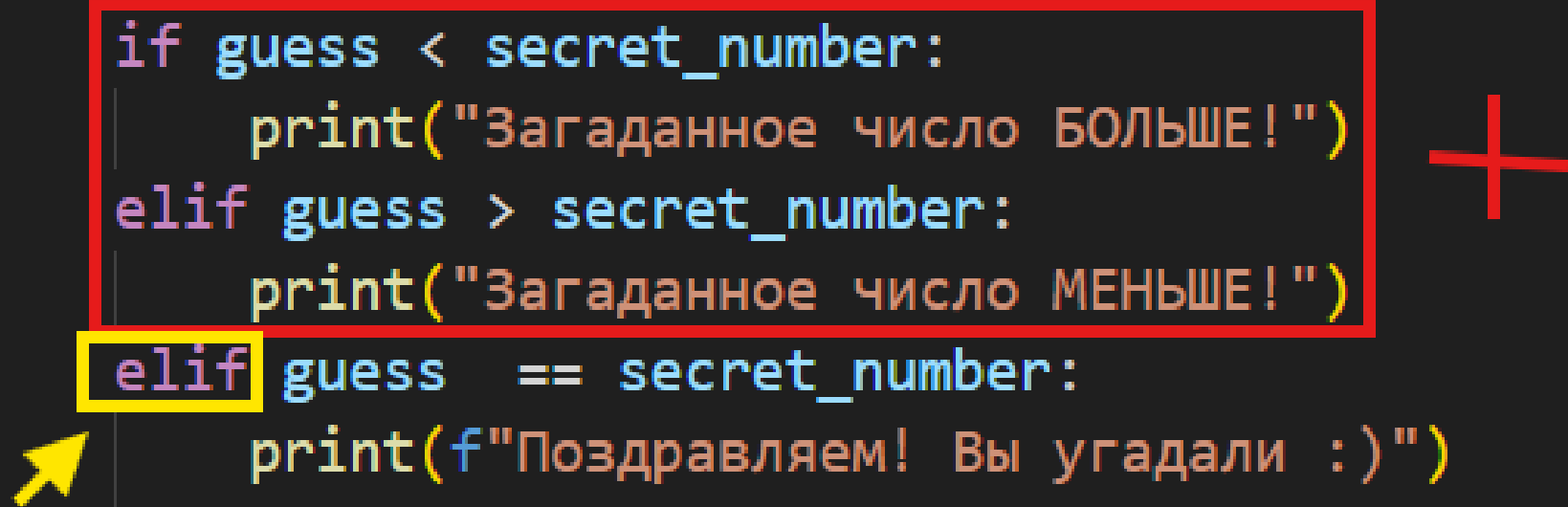
Усовершенствуем программу «Угадай число». Теперь угадываем случайное число.

```
2 import random
3
4 secret_number = random.randint(1, 10)
5 guess=None
6
7 print("Компьютер загадал число от 1 до 100. Попробуйте угадать!")
8 while True:
9     guess = int(input("Ваша догадка: "))
10    if guess == secret_number:
11        print(f"Поздравляем! Вы угадали :)")
12        break
13    else:
14        print("Похоже вы не угадали :( Попробуйте ещё раз!")
```

Немного практики 3

Добавим небольшую подсказку в игру. Больше или меньше загаданное число.

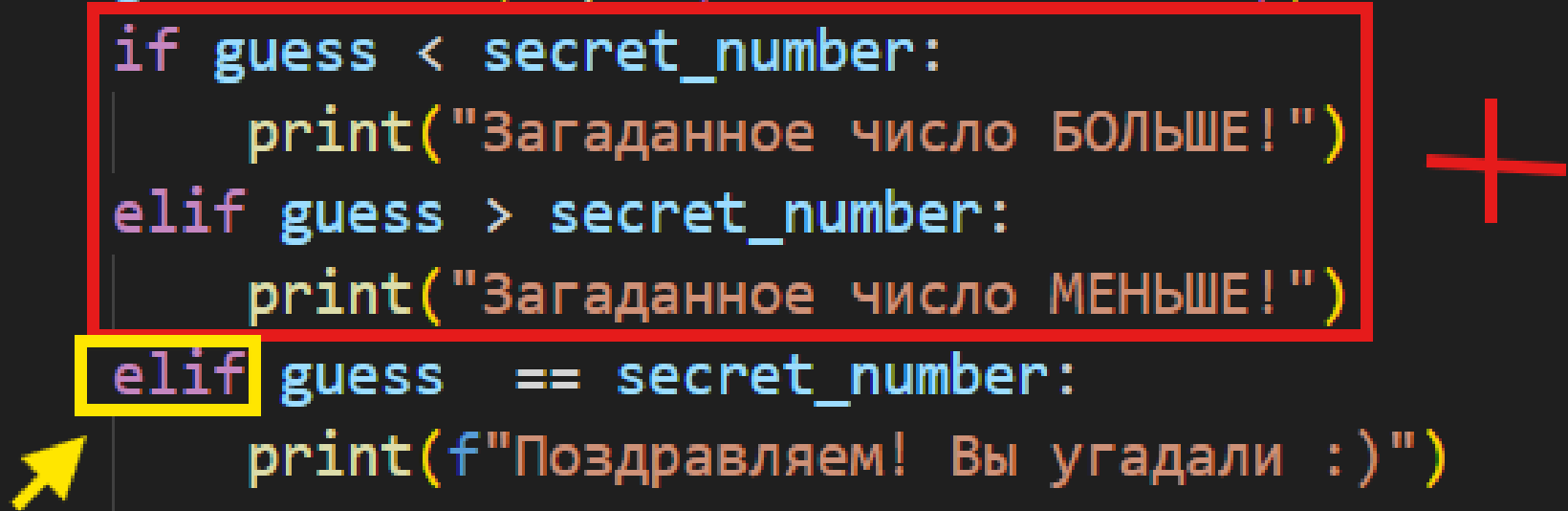
```
while True:
    guess = int(input("Ваша догадка: "))
    if guess < secret_number:
        print("Загаданное число БОЛЬШЕ!")
    elif guess > secret_number:
        print("Загаданное число МЕНЬШЕ!")
    elif guess == secret_number:
        print(f"Поздравляем! Вы угадали :)")
        break
    else:
        print("Похоже вы не угадали :( Попробуйте ещё раз!")
```



Немного практики 3

Добавим небольшую подсказку в игру. Больше или меньше загаданное число.

```
while True:
    guess = int(input("Ваша догадка: "))
    if guess < secret_number:
        print("Загаданное число БОЛЬШЕ!")
    elif guess > secret_number:
        print("Загаданное число МЕНЬШЕ!")
    elif guess == secret_number:
        print(f"Поздравляем! Вы угадали :)")
        break
    else:
        print("Похоже вы не угадали :( Попробуйте ещё раз!")
```



Немного практики 3

Добавим ограниченное количество попыток

```
guess=None
attempts = 0
max_attempts = 3

print("Компьютер загадал число от 1 до 100. Попробуйте угадать!")
while True:
    guess = int(input("Ваша догадка: "))
    attempts += 1

    if attempts >= max_attempts:
        print(f"Попытки закончились! Загаданное число было {secret_number}.")
        break

    if guess < secret_number:
        print("Загаданное число БОЛЬШЕ!")
    elif guess > secret_number:
        print("Загаданное число МЕНЬШЕ!")
    elif guess == secret_number:
        print(f"Поздравляем! Вы угадали за {attempts} попыток!")
        break
```

Спасибо за внимание!

